Deliverable 3 — Requirements Document

Healthcare Web Application

Course Code — CSE 616
Course Name — Web Engineering Lab

Group Number: 14

Group Members:
Israt Zannat — 22701027
Sajid Shahriar Sanam — 22701031
Shajeda Isnat — 22701060

Course Instructor:
Professor Rokan Uddin Faruqui

Department of Computer Science and Engineering,
University of Chittagong

Date Prepared: September 17, 2025

Contents

1 Introduction **5**

2 Backend Architecture Overview **6**
2.1 Technology Stack . . . 7
2.2 Backend Project Structure . . . 7
2.3 Backend Architecture Diagram . . . 8

3 RESTful API Implementation **9**
3.1 Implemented API Endpoints . . . 9
3.2 Example API Request and Response . . . 10
3.2.1 User Registration . . . 10
3.2.2 User Login . . . 10
3.2.3 Create Appointment . . . 11
3.3 API Testing Using Postman . . . 11

4 Authentication and Authorization System **12**
4.1 Authentication Process . . . 12
4.2 JWT-Based Authentication . . . 13
4.3 Password Security . . . 13
4.4 Role-Based Authorization . . . 13
4.5 Implemented Security Mechanisms . . . 14
4.6 Authentication Workflow Diagram . . . 14

5 Database Models and Schema Design **15**
5.1 User Model . . . 16
5.2 Appointment Model . . . 16
5.3 Database Interaction Using Mongoose . . . 17
5.4 Example MongoDB Documents . . . 17
5.4.1 Example User Document . . . 17
5.4.2 Example Appointment Document . . . 18
5.5 Data Model Overview . . . 18

6 Unit and Integration Testing **19**
6.1 Testing Framework . . . 19
6.2 Implemented Test Cases . . . 19
6.3 Example Test Output . . . 19
6.4 Code Coverage Report . . . 20

7 API Documentation **21**
7.1 Base URL . . . 21
7.2 Authentication Mechanism . . . 22
7.3 Implemented API Endpoints . . . 22
7.4 Example API Request . . . 22
7.5 Example API Response . . . 22
7.6 API Testing Using Postman . . . 23

8 Error Handling and Logging **23**
8.1 Error Handling Strategy 24
8.2 HTTP Status Codes 24
8.3 Logging Mechanism 24
8.4 Benefits of Error Handling and Logging 25

9 Conclusion **25**

10 References **26**

1 Introduction

This document presents Deliverable 3 of the Healthcare Web Application project for CSE 616: Web Engineering Lab. The focus of this deliverable is the implementation of the **Backend Minimum Viable Product (MVP)**, which transforms the system from design-level planning into an operational server-side application.

In Deliverable 1, the project requirements, actors, functional requirements, non-functional requirements, and technology stack were defined. In Deliverable 2, the system architecture, database design, API specifications, UI/UX mockups, and security architecture were prepared as the blueprint for development. Based on those foundations, Deliverable 3 implements the core backend modules required to support secure and structured communication between the frontend and the database.

The backend has been developed using **Node.js** and **Express.js**, with **MongoDB** and **Mongoose** used for data storage and schema management. The system provides RESTful API endpoints for key healthcare operations, including user registration, login, role-based access control, and appointment management. In addition, authentication has been implemented using **JSON Web Tokens (JWT)**, while password security has been ensured using **bcrypt**-based hashing.

This deliverable also includes database model implementation, protected route handling, role-based authorization, unit and integration testing, API documentation, and a test coverage report. These components collectively establish the technical foundation needed for the next phase of the project, where frontend integration and complete system interaction will be developed.

The main objectives of Deliverable 3 are as follows:

- To implement a functional backend server using Node.js and Express.js.
- To establish secure user authentication and role-based authorization.
- To design and implement the core database models required for MVP functionality.
- To provide RESTful API endpoints for user and appointment operations.
- To verify backend functionality through automated testing and coverage analysis.
- To prepare the system for frontend integration in the next deliverable.

Overall, Deliverable 3 represents the successful realization of the backend core of the Healthcare Web Application. It converts the prior requirement and design documents into a working software component that supports security, modularity, maintainability, and future system expansion.

Component	Description
Authentication Module	User registration and login using JWT
Authorization Module	Role-based access for patient, doctor, and admin
Database Models	User and Appointment schemas using Mongoose
Appointment API	Booking, viewing, and status update endpoints
Testing Framework	Automated tests using Jest and Supertest
API Documentation	Endpoint descriptions with request/response formats

Table 1: Core Backend MVP Components Implemented in Deliverable 3

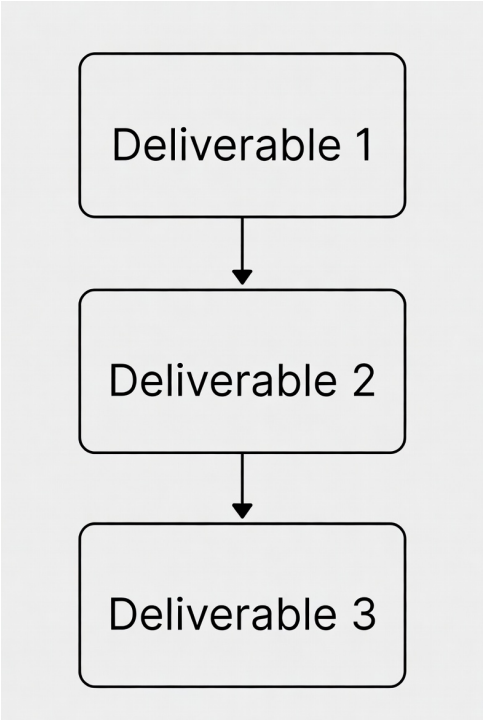

Figure 1: Development progression from requirements analysis to design and backend implementation.

2 Backend Architecture Overview

The backend of the Healthcare Web Application is designed using a modular and layered architecture. The backend acts as the intermediary between the frontend interface and the database, ensuring that user requests are processed securely and efficiently. It handles authentication, business logic, data validation, and database interactions.

The backend is implemented using **Node.js** and **Express.js**, which provide a lightweight and scalable server framework. Data is stored in **MongoDB**, a NoSQL document-oriented database, and schema definitions are handled using the **Mongoose** Object Data Modeling (ODM) library.

The architecture separates different concerns of the application into independent modules, improving maintainability and scalability. Each module performs a specific role in the system, such as request routing, authentication, database modeling, or business logic processing.

The interaction flow of the backend system can be summarized as follows:

1. The client (frontend or API testing tool such as Postman) sends an HTTP request.
2. The request is received by the Express server.
3. The request is routed to the appropriate API endpoint.
4. Middleware components handle authentication and request validation.
5. The controller processes the business logic.
6. The controller interacts with database models using Mongoose.
7. The server returns a JSON response to the client.

2.1 Technology Stack

The main technologies used in the backend implementation are listed in Table 2.

Technology	**Purpose**
Node.js	JavaScript runtime environment used to run the backend server
Express.js	Web framework for building RESTful APIs
MongoDB	NoSQL database used to store application data
Mongoose	Object Data Modeling (ODM) library for MongoDB
JWT (JSON Web Token)	Authentication mechanism for secure API access
bcrypt	Password hashing for secure credential storage
Jest	Testing framework for unit and integration tests
Supertest	HTTP testing library for API endpoint testing

Table 2: Backend Technology Stack

2.2 Backend Project Structure

The backend follows a modular folder structure where different components of the application are separated according to their functionality.

```
src/
  config
```

```
   db.js
controllers
   authController.js
   appointmentController.js
   userController.js
middleware
   auth.js
models
   User.js
   Appointment.js
routes
   authRoutes.js
   userRoutes.js
   appointmentRoutes.js
   adminRoutes.js
utils
   generateToken.js
app.js
server.js
```

Each folder in the structure has a specific responsibility:

- **config** contains database configuration and connection logic.
- **controllers** implement the core business logic of the application.
- **middleware** provides authentication and request validation logic.
- **models** define database schemas using Mongoose.
- **routes** define API endpoints and route incoming requests to controllers.
- **utils** contains reusable helper functions such as JWT token generation.

2.3 Backend Architecture Diagram

Figure 2 illustrates the backend request processing pipeline. It shows how a client request travels through the API server, authentication middleware, controllers, database models, and finally the MongoDB database before returning a response.

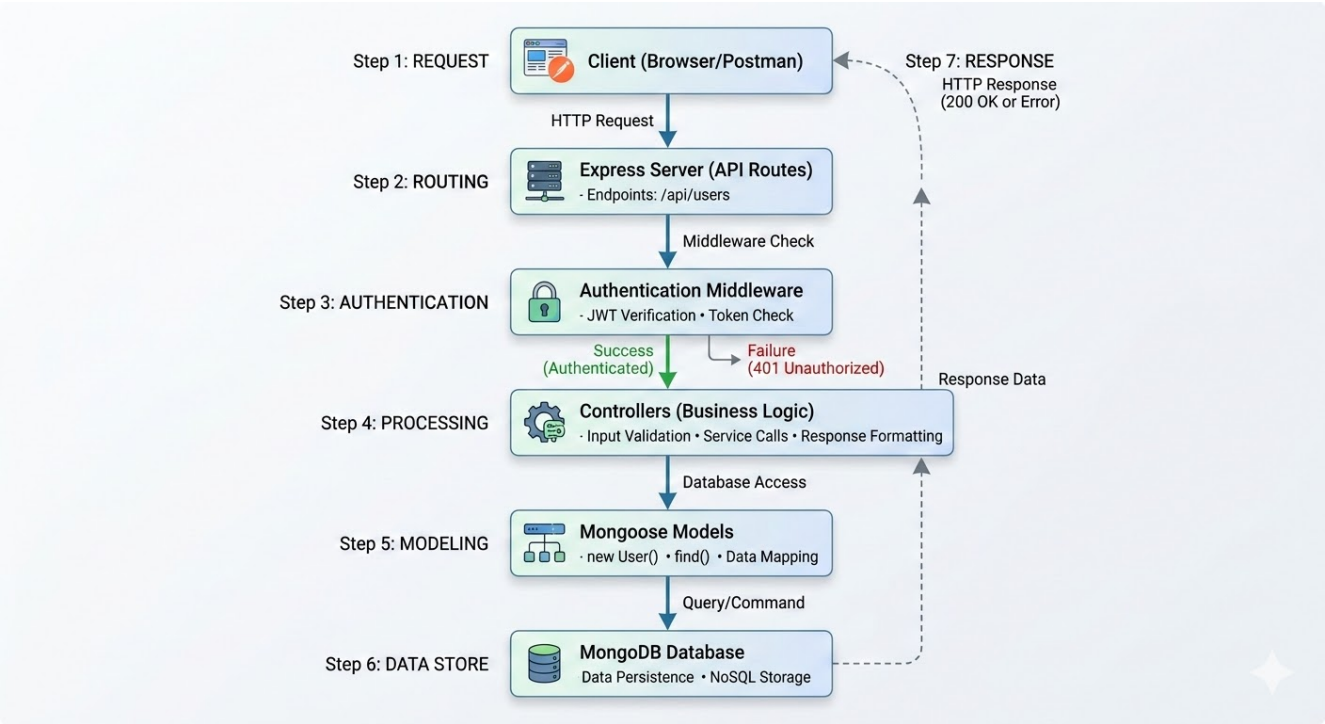

Figure 2: Backend architecture and request processing pipeline of the Healthcare Web Application.

3 RESTful API Implementation

The Healthcare Web Application backend exposes a set of RESTful API endpoints that allow clients to interact with the system. These APIs enable operations such as user registration, login, profile retrieval, and appointment management.

The backend follows REST architectural principles where resources are accessed using standard HTTP methods such as **GET**, **POST**, and **PATCH**. All communication between the client and the server occurs through **JSON-formatted requests and responses**.

Each API endpoint is organized using Express.js routing modules. Incoming requests are validated, processed by controller functions, and then executed through database models using Mongoose.

3.1 Implemented API Endpoints

Table 8 summarizes the main API endpoints implemented in the Backend MVP.

Method	Endpoint	Description
POST	/api/v1/auth/register	Register a new user account
POST	/api/v1/auth/login	Authenticate user and generate JWT token
GET	/api/v1/users/me	Retrieve profile information of the authenticated user
POST	/api/v1/appointments	Create a new appointment request
GET	/api/v1/appointments/me	Retrieve appointments associated with the user
PATCH	/api/v1/appointments/:id/status	Update appointment status (doctor/admin)

Table 3: RESTful API Endpoints Implemented

3.2 Example API Request and Response

The following examples illustrate how the API endpoints are used.

3.2.1 User Registration

This endpoint allows a new patient to create an account in the system.

Endpoint:

```
POST /api/v1/auth/register
```

Request Body:

```
{
  "name": "Test Patient",
  "email": "patient1@test.com",
  "password": "password123",
  "role": "patient"
}
```

Response Example:

```
{
  "success": true,
  "token": "JWT_TOKEN"
}
```

3.2.2 User Login

This endpoint verifies user credentials and generates an authentication token.

Endpoint:

```
POST /api/v1/auth/login
```

Request Body:

```
{
  "email": "patient1@test.com",
  "password": "password123"
}
```

Response Example:

```
{
  "success": true,
  "token": "JWT_TOKEN"
}
```

3.2.3 Create Appointment

This endpoint allows a patient to create an appointment with a doctor.

Endpoint:

```
POST /api/v1/appointments
```

Request Body:

```
{
  "doctorId": "DOCTOR_ID",
  "dateTime": "2026-02-22T10:00:00.000Z",
  "reason": "Fever and headache"
}
```

3.3 API Testing Using Postman

The implemented APIs were tested using Postman to verify correct request handling, authentication, and response generation.

Figure 8 shows an example of successful API testing through Postman.

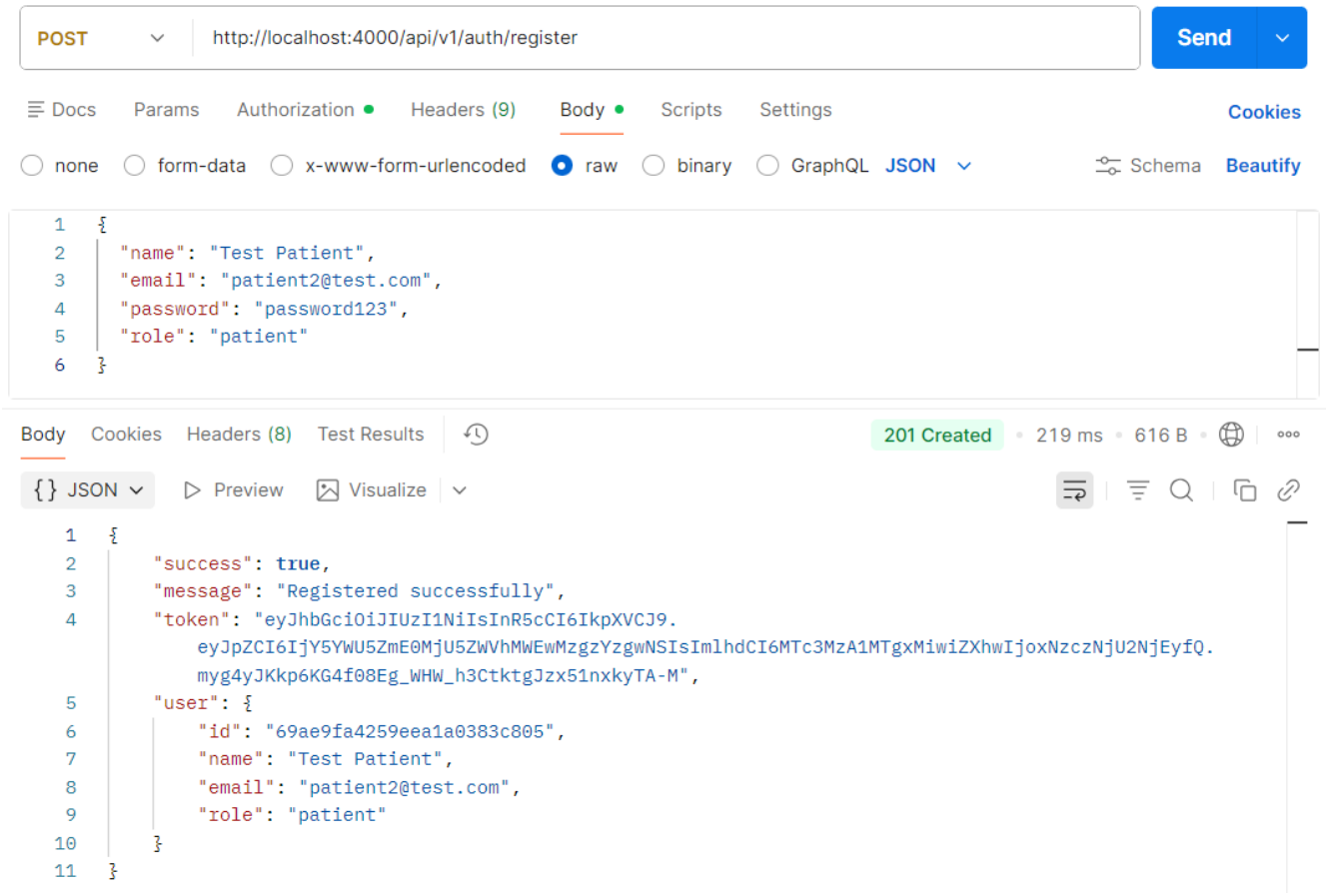

Figure 3: Testing RESTful API endpoints using Postman

4 Authentication and Authorization System

Security is a critical requirement in the Healthcare Web Application because the system handles sensitive user and medical information. To protect backend resources, the application implements both **authentication** and **authorization** mechanisms.

Authentication is used to verify the identity of a user, while authorization determines what actions that authenticated user is allowed to perform in the system. In the backend implementation, authentication is achieved using **JSON Web Tokens (JWT)**, and authorization is enforced through **Role-Based Access Control (RBAC)**.

4.1 Authentication Process

When a user registers, the backend stores the user's information in the database after securely hashing the password using bcrypt. During login, the server validates the email and password combination. If the credentials are correct, the backend generates a signed JWT token and sends it to the client.

For every protected API request, the client includes this token in the HTTP Authorization header. The server verifies the token before granting access to the requested resource.

The overall authentication flow can be summarized as follows:

1. A user submits login credentials.

2. The backend validates the credentials against the database.
3. If valid, the backend generates a JWT token.
4. The client stores the token and sends it with protected requests.
5. The authentication middleware verifies the token for each request.
6. If verification succeeds, access is granted; otherwise, the request is rejected.

4.2 JWT-Based Authentication

JSON Web Token (JWT) is used to maintain stateless authentication. This approach eliminates the need for traditional session storage on the server and makes the backend more scalable.

After successful login or registration, the backend generates a token containing the user identifier and authentication metadata. The token is signed using a secret key defined in the environment variables.

A protected request must include the following header:

```
Authorization: Bearer <JWT_TOKEN>
```

If the token is missing, malformed, or invalid, the backend returns an unauthorized response.

4.3 Password Security

Passwords are not stored in plain text. Instead, the backend uses **bcrypt** to hash user passwords before they are saved in the database. This ensures that even if the database is compromised, the original passwords cannot be easily retrieved.

The password hashing process is implemented in the `User` model using a pre-save hook. During login, bcrypt compares the entered password with the stored hashed password.

This approach improves system security by protecting user credentials and following secure authentication practices.

4.4 Role-Based Authorization

The backend enforces role-based access control to restrict access to resources based on user roles. Each user in the system is assigned one of the following roles:

- **Patient**
- **Doctor**
- **Admin**

After the authentication middleware verifies the token, authorization middleware checks the role of the authenticated user before allowing access to certain API endpoints.

For example:

- A patient can create appointments.

- A doctor can view assigned appointments and update appointment status.
- An admin can access admin-only routes and system-level operations.

If a user attempts to access a route without the required role, the backend returns a **403 Forbidden** response.

Role	**Permitted Actions**
Patient	Register, login, view own profile, create appointments, view own appointments
Doctor	Login, view assigned appointments, update appointment status
Admin	Login, access admin-only routes, perform system-level operations

Table 4: Role-Based Access Permissions

4.5 Implemented Security Mechanisms

The authentication and authorization system includes the following implemented security features:

- JWT-based token authentication
- Password hashing using bcrypt
- Middleware-based route protection
- Role-based access control for restricted endpoints
- Unauthorized request rejection using HTTP 401 responses
- Forbidden access rejection using HTTP 403 responses

4.6 Authentication Workflow Diagram

Figure 4 illustrates the authentication and authorization workflow of the backend system.

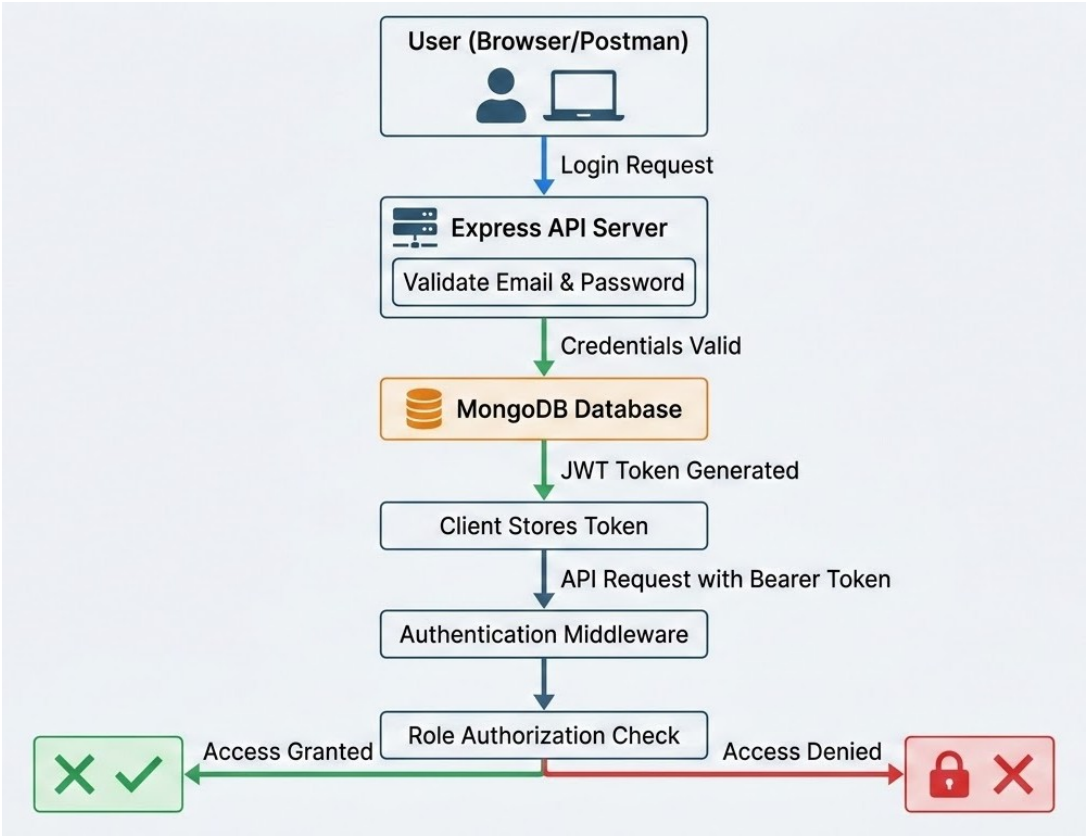

Figure 4: Authentication and authorization workflow using JWT and role-based access control.

Overall, the implemented authentication and authorization system provides a secure and scalable mechanism for controlling access to backend resources. It ensures that only verified users can access the system and that each user can perform only the actions permitted by their assigned role.

5 Database Models and Schema Design

The Healthcare Web Application backend uses **MongoDB** as its primary database system. MongoDB is a document-oriented NoSQL database that stores data in flexible JSON-like documents. This structure allows the application to handle dynamic data structures while maintaining high performance and scalability.

To interact with MongoDB, the backend uses **Mongoose**, which is an Object Data Modeling (ODM) library for Node.js. Mongoose provides schema definitions, validation mechanisms, and convenient methods for interacting with the database.

For the Backend MVP, two core database models were implemented:

- User Model
- Appointment Model

These models represent the primary entities required for authentication and appointment management within the healthcare system.

5.1 User Model

The User model represents individuals who can access the system. Each user account stores basic identity information, login credentials, and the role assigned to that user.

The roles supported by the system include:

- Patient
- Doctor
- Admin

Table 5 describes the fields of the User schema.

Field Name	**Data Type**	**Description**
name	String	Name of the user
email	String	Unique email address used for login
passwordHash	String	Hashed password stored securely using bcrypt
role	String	Defines whether the user is a patient, doctor, or admin
createdAt	Date	Timestamp of account creation
updatedAt	Date	Timestamp of last account update

Table 5: User Schema Description

The password field is hashed before being stored in the database using bcrypt to enhance security. This ensures that user credentials are protected even if database access is compromised.

5.2 Appointment Model

The Appointment model represents a scheduled interaction between a patient and a doctor. It allows patients to request appointments and enables doctors or administrators to manage appointment status.

Each appointment record links a patient to a doctor and stores relevant scheduling information.

Table 6 describes the structure of the Appointment schema.

Field Name	Data Type	Description
patientId	ObjectId	Reference to the patient user
doctorId	ObjectId	Reference to the doctor user
dateTime	Date	Scheduled appointment time
reason	String	Reason for the appointment
status	String	Status of appointment (pending, approved, rejected)
createdAt	Date	Timestamp of appointment creation
updatedAt	Date	Timestamp of last appointment update

Table 6: Appointment Schema Description

The patientId and doctorId fields reference user records in the User collection, enabling relationships between patients and doctors within the system.

5.3 Database Interaction Using Mongoose

Mongoose provides convenient methods for interacting with the database. Controllers use these models to perform operations such as:

- Creating new users
- Authenticating existing users
- Creating appointment requests
- Retrieving user-specific appointments
- Updating appointment status

These operations ensure that the backend maintains structured and secure access to the healthcare system's data.

5.4 Example MongoDB Documents

To further illustrate how data is stored in the database, sample MongoDB documents for the implemented collections are shown below.

These example documents reflect the actual structure used in the backend implementation and demonstrate how Mongoose schemas are translated into MongoDB records.

5.4.1 Example User Document

The following example shows how a user record is stored in the `User` collection.

```
{
  "_id": "699a0d74c1fd24636cc5e106",
  "name": "Test Patient",
  "email": "patient1@test.com",
```

```
  "passwordHash": "$2b$10$8QwzY....hashedValue....",
  "role": "patient",
  "createdAt": "2026-02-22T10:00:00.000Z",
  "updatedAt": "2026-02-22T10:00:00.000Z"
}
```

5.4.2 Example Appointment Document

The following example shows how an appointment record is stored in the `Appointment` collection.

```
{
  "_id": "699a1b30b4d8e761d7d4a321",
  "patientId": "699a0d74c1fd24636cc5e106",
  "doctorId": "699a1025b1fd24636cc5e245",
  "dateTime": "2026-02-22T10:00:00.000Z",
  "reason": "Fever and headache",
  "status": "pending",
  "createdAt": "2026-02-22T10:05:00.000Z",
  "updatedAt": "2026-02-22T10:05:00.000Z"
}
```

5.5 Data Model Overview

Figure 5 illustrates the relationship between the primary entities in the database.

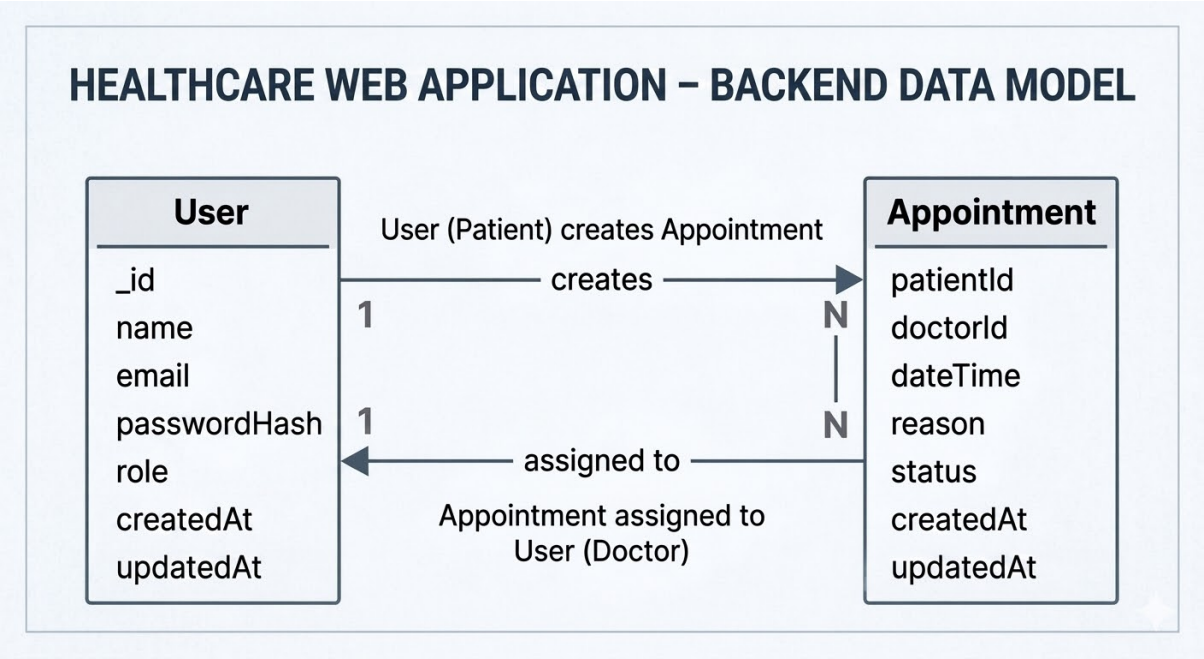

Figure 5: Simplified data model showing the relationship between User and Appointment entities.

The database structure supports the core functionality required for the Backend MVP while allowing future expansion for additional healthcare features such as medical records, prescriptions, and notifications.

To ensure reliability and correctness of the backend system, automated unit and integration testing were performed using modern JavaScript testing tools.

6 Unit and Integration Testing

Testing is an essential step in backend development to ensure that system components function correctly and reliably. In the Healthcare Web Application backend, automated testing was implemented using the **Jest** testing framework together with **Supertest** for HTTP request testing.

Jest provides a simple and powerful testing environment for Node.js applications, while Supertest allows developers to simulate HTTP requests to API endpoints and verify the responses returned by the server.

6.1 Testing Framework

The following tools were used for backend testing:

Tool	**Purpose**
Jest	JavaScript testing framework used to write and execute automated tests
Supertest	Library used to test HTTP endpoints of the Express server
MongoDB Test Database	Separate database used during test execution

Table 7: Testing Tools Used in the Backend

These tools allow the backend APIs to be tested without manually interacting with the application.

6.2 Implemented Test Cases

Two primary authentication test cases were implemented to verify the correctness of the backend authentication system:

- **User Registration Test** – verifies that a new user can successfully register in the system.
- **User Login Test** – verifies that an existing user can log in and receive a valid JWT token.

These tests confirm that the authentication module correctly handles user credentials, generates tokens, and returns appropriate responses.

6.3 Example Test Output

Figure 6 shows the output of the automated tests executed using the Jest testing framework.

```
C:\Users\Asus>cd C:\Users\Asus\healthcare-backend

C:\Users\Asus\healthcare-backend>npm test

> healthcare-backend@1.0.0 test
> set NODE_ENV=test&& node --experimental-vm-modules node_modules/jest/bin/jest.js --config jest.config.cjs --runInBand

  console.log
    [dotenv@17.3.1] injecting env (5) from .env -- tip: ⚙  override existing env vars with { override: true }

      at _log (node_modules/dotenv/lib/main.js:139:11)

(node:19060) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
  console.log
    ✅ MongoDB connected

      at log (src/config/db.js:11:13)

 PASS  tests/auth.test.cjs
  Auth API
    √ should register a new patient (256 ms)
    √ should login an existing user (282 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        3.341 s
Ran all test suites.
```

Figure 6: Successful execution of authentication test cases using Jest

The results indicate that both implemented test cases passed successfully.

6.4 Code Coverage Report

In addition to executing test cases, a coverage report was generated to measure how much of the backend codebase is exercised by the tests.

The coverage report was generated using the following command:

```
npm run test:coverage
```

The coverage report evaluates the following metrics:

- Statements coverage
- Branch coverage
- Function coverage
- Line coverage

Figure 7 shows the generated coverage report.

```
 PASS  tests/auth.test.cjs
  Auth API
    √ should register a new patient (204 ms)
    √ should login an existing user (228 ms)

----------------------------|---------|----------|---------|---------|--------------------
File                        | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------------|---------|----------|---------|---------|--------------------
All files                   |   44.85 |    22.58 |   38.88 |   46.21 |
 src                        |   42.85 |        0 |       0 |   42.85 |
  app.js                    |      75 |        0 |       0 |      75 | 16,26-27
  server.js                 |       0 |        0 |       0 |       0 | 5-19
 src/config                 |   71.42 |       50 |     100 |   71.42 |
  db.js                     |   71.42 |       50 |     100 |   71.42 | 13-14
 src/controllers            |   34.32 |     27.5 |   33.33 |   35.93 |
  appointmentController.js  |    7.69 |        0 |       0 |    8.33 | 6-38,44-58,64-95
  authController.js         |   73.07 |     64.7 |     100 |   73.07 | 9,16,37,46,53,58,70
  userController.js         |      50 |      100 |       0 |      50 | 2
 src/middleware             |   16.66 |        0 |   33.33 |   16.66 |
  auth.js                   |   16.66 |        0 |   33.33 |   16.66 | 5-24,30-33
 src/models                 |   88.88 |       50 |     100 |     100 |
  Appointment.js            |     100 |      100 |     100 |     100 |
  User.js                   |    87.5 |       50 |     100 |     100 | 40
 src/routes                 |   91.66 |      100 |       0 |   91.66 |
  adminRoutes.js            |   66.66 |      100 |       0 |   66.66 | 11
  appointmentRoutes.js      |     100 |      100 |     100 |     100 |
  authRoutes.js             |     100 |      100 |     100 |     100 |
  userRoutes.js             |     100 |      100 |     100 |     100 |
 src/utils                  |     100 |       50 |     100 |     100 |
  generateToken.js          |     100 |       50 |     100 |     100 | 5
----------------------------|---------|----------|---------|---------|--------------------
Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        2.762 s, estimated 4 s
Ran all test suites.
```

Figure 7: Code coverage report generated from automated backend tests

The coverage report confirms that the authentication and routing components of the backend are tested successfully.

Overall, the implemented automated testing framework improves backend reliability and helps ensure that critical functionality operates correctly as the system evolves.

7 API Documentation

The Healthcare Web Application backend exposes a set of RESTful API endpoints that allow clients such as web applications or mobile applications to interact with the system.

The API follows standard REST principles where resources are accessed using HTTP methods such as **GET**, **POST**, **PUT**, and **DELETE**. All API responses are returned in **JSON format**.

7.1 Base URL

All API endpoints are accessible through the following base URL:

`http://localhost:4000/api/v1`

Each service module extends this base URL with specific resource paths.

7.2 Authentication Mechanism

Protected endpoints require authentication using **JSON Web Tokens (JWT)**. After successful login, the server generates a JWT token which must be included in subsequent requests.

The token is sent in the HTTP request header using the following format:

```
Authorization: Bearer <JWT_TOKEN>
```

The authentication middleware verifies the token before allowing access to protected routes.

7.3 Implemented API Endpoints

Table 8 lists the primary API endpoints implemented in the Backend MVP.

Method	Endpoint	Description
POST	/auth/register	Register a new user account
POST	/auth/login	Authenticate user and return JWT token
POST	/appointments	Create a new appointment
GET	/appointments	Retrieve appointments for the logged-in user
PUT	/appointments/:id	Update appointment status

Table 8: Implemented API Endpoints

7.4 Example API Request

The following example demonstrates a request to register a new user.

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "name": "Test Patient",
  "email": "patient1@test.com",
  "password": "password123",
  "role": "patient"
}
```

7.5 Example API Response

If the request is successful, the server returns a JSON response similar to the following:

```
{
  "success": true,
  "message": "User registered successfully",
  "user": {
```

```
    "id": "699a0d74c1fd24636cc5e106",
    "name": "Test Patient",
    "email": "patient1@test.com",
    "role": "patient"
  }
}
```


7.6 API Testing Using Postman

The implemented API endpoints were tested using **Postman**. Postman allows developers to send HTTP requests and inspect responses to verify backend functionality.

Figure 8 shows a successful API request executed using Postman.

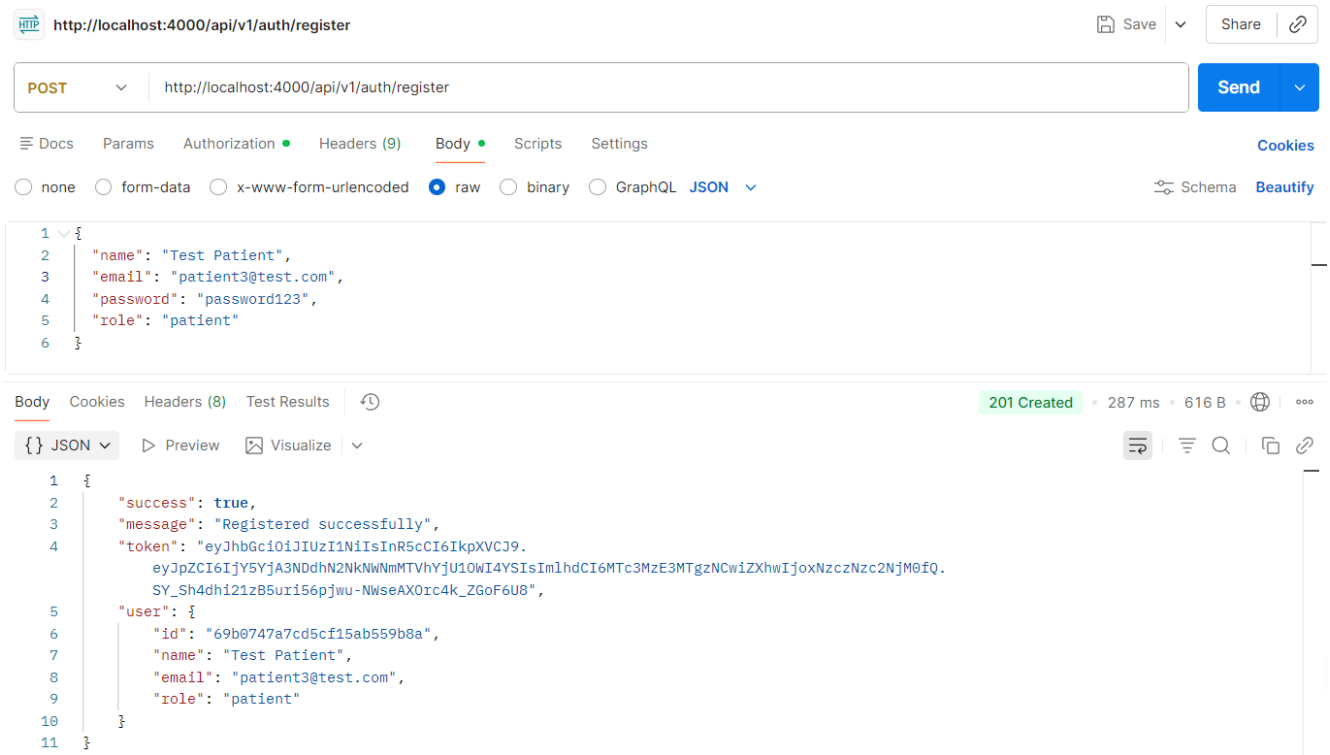

Figure 8: Testing API endpoints using Postman

These tests confirm that the backend API correctly processes requests and returns valid responses for client applications.

8 Error Handling and Logging

Reliable backend systems must properly handle unexpected errors and provide meaningful feedback to clients. In the Healthcare Web Application backend, a structured error handling and logging mechanism was implemented to improve system stability and debugging capability.

8.1 Error Handling Strategy

The backend follows a centralized error handling strategy using Express middleware. When an error occurs in any controller or service, it is forwarded to a global error handler that generates a standardized JSON response.

Typical API error responses follow this format:

```
{
  "success": false,
  "message": "Error description"
}
```

This approach ensures that clients always receive consistent error messages regardless of where the error occurs in the application.

8.2 HTTP Status Codes

The backend uses appropriate HTTP status codes to indicate the result of each request. Some commonly used status codes include:

Status Code	**Meaning**
200 OK	Request processed successfully
201 Created	Resource successfully created (e.g., user registration)
400 Bad Request	Invalid input data or malformed request
401 Unauthorized	Authentication token missing or invalid
404 Not Found	Requested resource does not exist
500 Internal Server Error	Unexpected server error

Table 9: HTTP Status Codes Used in the Backend

Using proper status codes helps client applications correctly interpret API responses.

8.3 Logging Mechanism

Basic logging is implemented to record important backend events. Logging messages are generated for events such as:

- Successful database connections
- API request processing
- Authentication events
- Server errors

For example, when the server successfully connects to MongoDB, a log message similar to the following is printed:

```
MongoDB connected
```

These logs assist developers in monitoring the system and diagnosing issues during development and testing.

8.4 Benefits of Error Handling and Logging

Implementing structured error handling and logging provides several advantages:

- Improves backend reliability
- Simplifies debugging during development
- Provides consistent API responses
- Helps identify system failures quickly

Overall, the error handling and logging mechanisms ensure that the backend system behaves predictably and remains easier to maintain as the application grows.

9 Conclusion

This deliverable successfully implements the Backend Minimum Viable Product (MVP) for the Healthcare Web Application. Building upon the requirements defined in Deliverable 1 and the system design prepared in Deliverable 2, the backend architecture was developed to provide secure, scalable, and modular server-side functionality.

The implemented backend provides a set of RESTful API endpoints that support user registration, authentication, and appointment management. Security mechanisms were implemented using JSON Web Tokens (JWT) and password hashing with bcrypt to ensure secure access to protected resources. Role-based authorization was also introduced to restrict system functionality according to user roles such as patient, doctor, and administrator.

The database layer was designed using MongoDB with Mongoose schemas to manage user accounts and appointment records. The modular architecture of controllers, routes, middleware, and models allows the backend to remain organized and easily maintainable.

Automated testing was conducted using the Jest testing framework along with Supertest to validate API functionality. The generated test results and coverage reports confirm that the core authentication components operate correctly. In addition, API endpoints were manually verified using Postman to ensure proper request handling and response generation.

Basic error handling and logging mechanisms were implemented to improve backend reliability and facilitate debugging during development. These mechanisms ensure that the backend can gracefully handle unexpected situations and provide consistent responses to client applications.

Overall, the Backend MVP provides a stable foundation for future development. The system is now prepared for integration with frontend components in the next phase of the project, enabling the Healthcare Web Application to evolve into a fully functional end-to-end system.

10 References

1. Node.js Documentation. Available at: https://nodejs.org/en/docs/
2. Express.js Documentation. Available at: https://expressjs.com/
3. MongoDB Documentation. Available at: https://www.mongodb.com/docs/
4. Mongoose Documentation. Available at: https://mongoosejs.com/docs/
5. JSON Web Token (JWT) Introduction. Available at: https://jwt.io/introduction/
6. Jest Testing Framework Documentation. Available at: https://jestjs.io/docs/
7. Supertest Library Documentation. Available at: https://github.com/ladjs/supertest